

-



- [Feature Engineering](#)
- Using PQL in Features

On this page

Using PQL in Features

Was this page helpful? 

Pulse Query Language (PQL) is Pulse's query language that allows users to compute complex values based on input data fields.

This topic introduces the basic concepts of PQL.

To learn how to use PQL in some specific scenarios, read the following user guides:

- [Handling NULL values with PQL](#)
- [PQL in derived features](#)
- [Calling UDFs](#)

Usage

The following image shows an example of how PQL is used in a map operator to create a derived field in a plan.

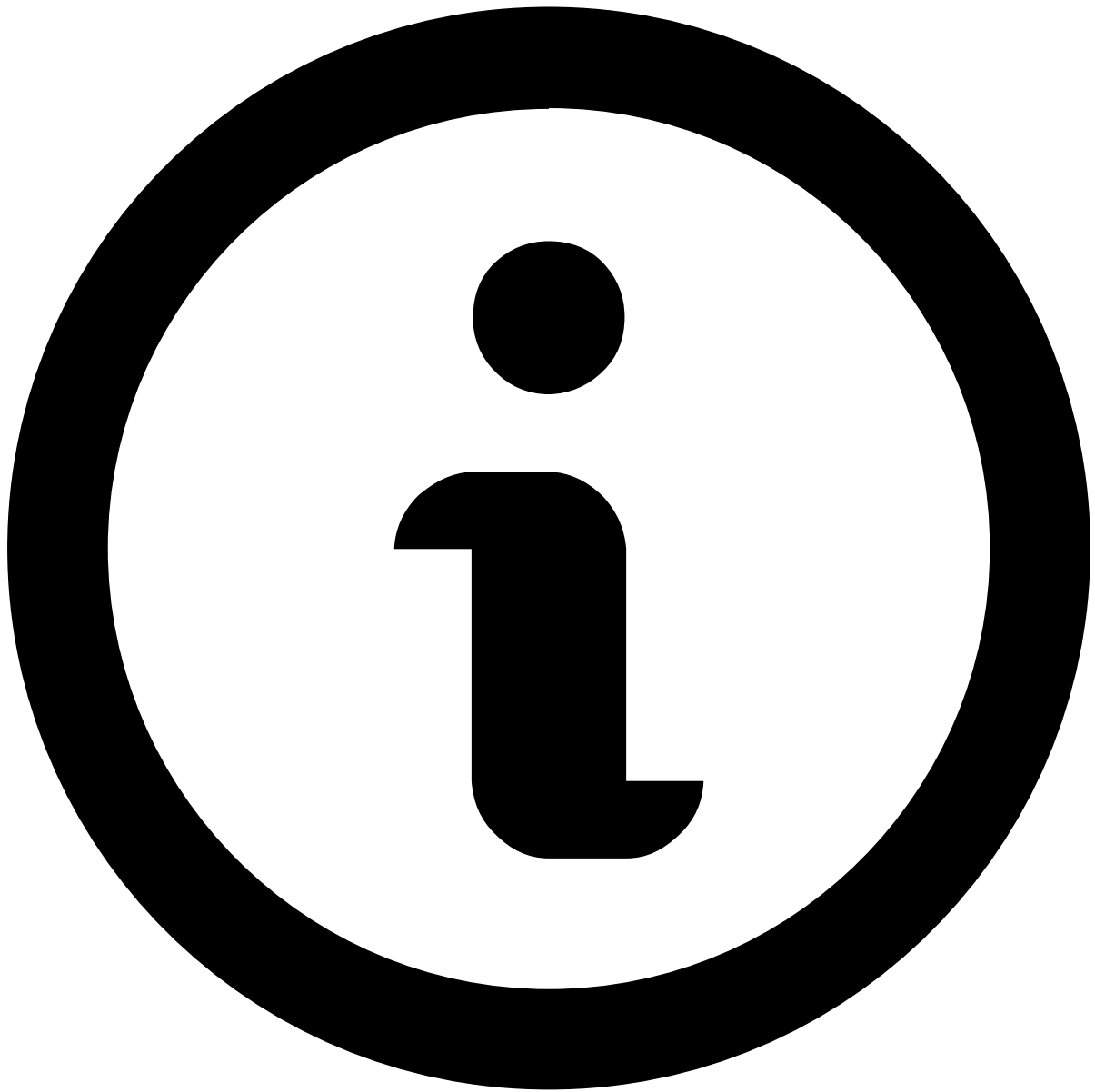
This map operator adds a categorical field called *Card Present Flag*, representing whether there was a bank card present in the transaction. The PQL expression is in the *Expression* column.

PQL expressions

In the Pulse UI, users can combine the following building blocks (values, operators, functions, subclauses) into PQL expressions:

Building block / subclause	Purpose, observation	Example(s)
Constants of primitive data types	Use a fixed value	1.999 'true' 'Swiped'
Input data fields	Use values of the input data. After typing the name of the data field, resolve the typed name to an existing data field name from the list that the UI offers.	Amount
: (i.e. the cast operator)	Convert data into another data type	Amount:int
Arithmetic operators	Calculate a new value from two values of int, long, float or double type	Balance - Amount
Boolean operators	Calculate a new true/false value	and or ! ==
Function calls	You can call two types of functions: built-in functions, and custom functions (aka. UDFs)	The following example calls the <i>between()</i> built-in function: between(Amount, 0, 200)
Conditional expressions	Return a value based on a condition	if Amount > 1000 then 'HighAmount' else 'LowAmount'
Variable declarations	Declare auxiliary variables that help you calculating other values	The following example shows how you define two variables (first_timestamp and last_timestamp) in order to calculate a third value (last_timestamp - first_timestamp) / 60*60*1000.0) (let first_timestamp = first().Timestamp ?? 0 in let last_timestamp = last().Timestamp ?? 0 in (last_timestamp - first_timestamp) / 60*60*1000.0)
Collections, tuples	Work on tuples, lists, sets, and maps of data instead of a single data field	

Building block / subclause	Purpose, observation	Example(s)
		In the following example, OrderItems is a list of orders. Each item of the list is a tuple with the following fields: ItemId, Quantity, Price <pre>OrderItems.get(0).Quantity</pre>
Aggregations, and functions over aggregated datasets	Aggregate data to calculate minimum, maximum, average, or alternatively to access specific records of the aggregated dataset	<pre>avg(Amount) - prev().Amount</pre>
?? (i.e. the coalesce operator)	Handle NULL values	<pre>Amount ?? 0</pre>



Reserved keywords

Some keywords and symbols cannot be used outside of their original purpose. See the list of [reserved keywords](#) for more information.